PostgreSQL 18

Lecciones prácticas compiladas

Tienda de electrónica · SQL · Administración · PL/pgSQL · Rendimiento

Edición: noviembre de 2026

Contenido: Compilación de las lecciones disponibles en esta conversación. La numeración original se conserva; no aparece una lección 20 en el hilo disponible.

Uso recomendado: Ejecuta los ejemplos en una base de pruebas PostgreSQL 18. Los objetos lN_* están pensados para evitar colisiones entre lecciones.

Índice temático

Arquitectura, administración y monitorización

- Lección 1: Arquitectura y rendimiento: entrada/salida asíncrona (AIO)
- Lección 13: Monitorización con pg_stat_activity
- Lección 14: MVCC, VACUUM y ANALYZE
- Lección 15: Transacciones, SAVEPOINT y RETURNING old/new
- Lección 23: Particionamiento por fecha y partition pruning

Consultas SQL, JOIN, subconsultas y CTE

- Lección 2: INNER JOIN y relaciones entre tablas
- Lección 3: LEFT JOIN, NULL y COALESCE
- Lección 4: Subconsultas correlacionadas con EXISTS
- Lección 5: CTE con WITH para dividir consultas complejas
- Lección 29: CTE recursiva para jerarquías

Funciones de ventana

- Lección 6: ROW_NUMBER(), RANK() y PARTITION BY
- Lección 7: Marcos ROWS y acumulados
- Lección 24: Marcos GROUPS y EXCLUDE

Índices, EXPLAIN y rendimiento

- Lección 8: Índices B-tree y lectura básica de EXPLAIN
- Lección 21: EXPLAIN ANALYZE: estimado vs real

Roles, permisos y seguridad

- Lección 9: Roles, permisos y mínimo privilegio
- Lección 22: Row-Level Security (RLS) en un SaaS

PL/pgSQL: funciones, procedimientos y triggers

- Lección 10: Funciones PL/pgSQL
- Lección 11: Procedimientos almacenados con CALL

Lección 12: Triggers para controlar el stock

Funciones y modelado específicos de PostgreSQL 18

Lección 16: Columnas generadas virtuales

Lección 17: UUIDv7: identificadores ordenables por tiempo

Lección 18: Periodos sin solapamiento con WITHOUT OVERLAPS

Lección 19: Claves foráneas temporales con PERIOD

Lección 25: COPY tolerante a errores: REJECT_LIMIT

Lección 26: Vistas materializadas para informes

Lección 27: Búsquedas Unicode con casefold()

Lección 28: Búsqueda aproximada con pg_trgm

Arquitectura, administración y monitorización

Este bloque agrupa las lecciones relacionadas para estudiar el tema de forma progresiva.

Lección 1 · Arquitectura y rendimiento: entrada/salida asíncrona (AIO)

↑ Volver al índice

Idea principal

PostgreSQL 18 incorpora una infraestructura de E/S asíncrona que permite solapar varias operaciones de lectura. El parámetro io_method controla el método de E/S y permite usar worker, sync o io_uring cuando está disponible.

Ejemplo ejecutable

```
SHOW server_version;
SHOW io_method;
SHOW effective_io_concurrency;

SELECT name, setting, unit, context
FROM pg_settings
WHERE name IN ('io_method','io_workers','effective_io_concurrency')
ORDER BY name;
```

Resultado esperado

Comprobación: Debes ver una versión 18.x. io_method mostrará el método activo; el valor exacto depende de cómo se haya compilado y configurado el servidor.

Ejercicio práctico

Explica qué diferencia práctica hay entre un método síncrono y uno asíncrono cuando PostgreSQL necesita leer muchos bloques del almacenamiento.

Solución explicada

En el método síncrono una operación espera a que termine la anterior. En AIO PostgreSQL puede emitir varias solicitudes y aprovechar mejor el dispositivo, reduciendo tiempos muertos. La mejora real depende del almacenamiento y del plan de ejecución.

PostgreSQL 18

Versión: Novedad de PostgreSQL 18: nueva infraestructura AIO y parámetros asociados. No toda consulta se acelera automáticamente.

Lección 13 · Monitorización con pg_stat_activity

↑ Volver al índice

Idea principal

pg_stat_activity muestra una fila por proceso de servidor y permite ver usuario, aplicación, estado, consulta y eventos de espera.

Ejemplo ejecutable

```
SET application_name='tienda_monitor';

SELECT pid,datname,usename,application_name,state,wait_event_type,wait_event
FROM pg_stat_activity
WHERE pid=pg_backend_pid();

SELECT pid,usename,now()-query_start AS duracion,wait_event_type,wait_event
FROM pg_stat_activity
WHERE state='active' AND pid<>pg_backend_pid()
ORDER BY query_start;
```

Resultado esperado

Comprobación: Tu sesión debe aparecer con application_name = tienda_monitor. Los demás resultados dependen de la actividad real del servidor.

Ejercicio práctico

Cuenta cuántos procesos bloquean tu sesión.

Solución explicada

SELECT cardinality(pg_blocking_pids(pg_backend_pid()));

PostgreSQL 18

Versión: pg_stat_activity no es nuevo. PostgreSQL 18 añade funciones/vistas relacionadas con NUMA cuando el servidor lo soporta.

Lección 14 · MVCC, VACUUM y ANALYZE

↑ Volver al índice

Idea principal

MVCC conserva versiones de filas para mantener consistencia entre transacciones. VACUUM permite reutilizar versiones muertas y ANALYZE actualiza estadísticas.

Ejemplo ejecutable

```
CREATE TEMP TABLE l14_productos(id int PRIMARY KEY,stock int);
INSERT INTO l14_productos SELECT n,n%20 FROM generate_series(1,10000) n;
DELETE FROM l14_productos WHERE id<=8000;
VACUUM (ANALYZE) l14_productos;
SELECT COUNT(*) AS filas FROM l14_productos;
SELECT reltuples::bigint FROM pg_class WHERE oid='l14_productos'::regclass;
```

Resultado esperado

Comprobación: COUNT(*) = 2000. reltuples será aproximadamente 2000 después de ANALYZE.

Ejercicio práctico

Elimina stock=0, ejecuta VACUUM ANALYZE y comprueba cuántas filas quedan.

Solución explicada

Quedarán 1900 filas. VACUUM normal no suele reducir el archivo en disco; deja el espacio reutilizable.

PostgreSQL 18

Versión: MVCC y VACUUM son históricos. PostgreSQL 18 habilita checksums de páginas por defecto en clústeres nuevos creados con initdb.

Lección 15 · Transacciones, SAVEPOINT y RETURNING old/new

↑ Volver al índice

Idea principal

BEGIN/COMMIT agrupan operaciones. SAVEPOINT permite deshacer solo una parte. PostgreSQL 18 amplía RETURNING para exponer valores old/new explícitamente.

Ejemplo ejecutable

```
CREATE TEMP TABLE l15_productos(id int PRIMARY KEY,nombre text,stock int);
INSERT INTO l15_productos VALUES (1,'Teclado',5);
BEGIN;
SAVEPOINT antes;
UPDATE l15_productos
SET stock=stock-2
WHERE id=1 AND stock>=2
RETURNING nombre,old.stock AS anterior,new.stock AS nuevo;
ROLLBACK TO SAVEPOINT antes;
SELECT * FROM l15_productos;
COMMIT;
```

Resultado esperado

Comprobación: RETURNING muestra 5 -> 3, pero después del ROLLBACK TO SAVEPOINT el stock vuelve a 5.

Ejercicio práctico

Crea una transacción que inserte un pedido, descuente stock y luego la cancele completamente.

Solución explicada

Usa BEGIN, realiza las operaciones y termina con ROLLBACK. Ningún cambio debe permanecer.

PostgreSQL 18

Versión: Novedad de PostgreSQL 18: referencias old/new explícitas en RETURNING para DML.

Lección 23 · Particionamiento por fecha y partition pruning

↑ Volver al índice

Idea principal

El particionamiento divide físicamente una tabla grande. El pruning elimina del plan las particiones imposibles según el filtro.

Ejemplo ejecutable

```
CREATE TABLE l23_pedidos(id int,fecha date,total numeric(10,2),PRIMARY KEY(id,fecha))
PARTITION BY RANGE(fecha);
CREATE TABLE l23_2026_01 PARTITION OF l23_pedidos FOR VALUES FROM ('2026-01-01') TO ('2026-02-
01');
CREATE TABLE l23_2026_02 PARTITION OF l23_pedidos FOR VALUES FROM ('2026-02-01') TO ('2026-03-
01');
INSERT INTO l23_pedidos VALUES (1,'2026-01-10',100),(2,'2026-02-10',200);
EXPLAIN (COSTS OFF) SELECT * FROM l23_pedidos WHERE fecha>='2026-02-01' AND fecha<'2026-03-
01';
```

Resultado esperado

Comprobación: El plan debe considerar solo la partición l23_2026_02 cuando enable_partition_pruning=on.

Ejercicio práctico

Añade una partición de marzo e inserta un pedido del 5 de marzo.

Solución explicada

Crea FOR VALUES FROM ('2026-03-01') TO ('2026-04-01'); inserta en la tabla padre y verifica tableoid.

PostgreSQL 18

Versión: El particionamiento ya existía; PostgreSQL 18 mejora planificación/costes para tablas con muchas particiones.

Consultas SQL, JOIN, subconsultas y CTE

Este bloque agrupa las lecciones relacionadas para estudiar el tema de forma progresiva.

Lección 2 · INNER JOIN y relaciones entre tablas

↑ Volver al índice

Idea principal

INNER JOIN combina filas que tienen coincidencia en ambas tablas. Es la base para reconstruir la información normalizada de clientes, pedidos, productos y líneas.

Ejemplo ejecutable

```
CREATE TEMP TABLE l2_clientes(cliente_id int PRIMARY KEY,nombre text);
CREATE TEMP TABLE l2_pedidos(pedido_id int PRIMARY KEY,cliente_id int REFERENCES l2_clientes);
CREATE TEMP TABLE l2_lineas(pedido_id int REFERENCES l2_pedidos,producto text,cantidad
int,precio numeric(10,2));
INSERT INTO l2_clientes VALUES (1,'Ana'),(2,'Luis');
INSERT INTO l2_pedidos VALUES (101,1),(102,2);
INSERT INTO l2_lineas VALUES (101,'Teclado',2,50),(101,'Ratón',1,25),(102,'Monitor',1,300);

SELECT c.nombre,p.pedido_id,SUM(l.cantidad*l.precio) AS total
FROM l2_pedidos p
JOIN l2_clientes c ON c.cliente_id=p.cliente_id
JOIN l2_lineas l ON l.pedido_id=p.pedido_id
GROUP BY c.nombre,p.pedido_id
ORDER BY p.pedido_id;
```

Resultado esperado

Comprobación: Ana | 101 | 125.00
Luis | 102 | 300.00

Ejercicio práctico

Añade una columna con el número total de unidades de cada pedido.

Solución explicada

Añade SUM(l.cantidad) AS unidades al SELECT. Como ya agrupamos por pedido, la suma se calcula dentro de cada pedido.

PostgreSQL 18

Versión: JOIN y claves foráneas no son novedades de PostgreSQL 18.

Lección 3 · LEFT JOIN, NULL y COALESCE

↑ Volver al índice

Idea principal

LEFT JOIN conserva todas las filas de la tabla izquierda aunque no exista coincidencia. COALESCE permite sustituir NULL por un valor alternativo, por ejemplo 0.

Ejemplo ejecutable

```
CREATE TEMP TABLE l3_clientes(id int PRIMARY KEY,nombre text);
CREATE TEMP TABLE l3_pedidos(id int PRIMARY KEY,cliente_id int REFERENCES l3_clientes,total
numeric(10,2));
INSERT INTO l3_clientes VALUES (1,'Ana'),(2,'Luis'),(3,'Marta');
INSERT INTO l3_pedidos VALUES (101,1,125),(102,2,300);

SELECT c.nombre,COUNT(p.id) AS pedidos,COALESCE(SUM(p.total),0) AS total
FROM l3_clientes c
LEFT JOIN l3_pedidos p ON p.cliente_id=c.id
GROUP BY c.id,c.nombre
ORDER BY c.id;
```

Resultado esperado

Comprobación: Ana | 1 | 125.00
Luis | 1 | 300.00
Marta | 0 | 0

Ejercicio práctico

Muestra todos los productos y sus unidades vendidas, incluyendo productos nunca vendidos con 0.

Solución explicada

Coloca productos a la izquierda, usa LEFT JOIN con líneas de pedido y COALESCE(SUM(cantidad),0).

PostgreSQL 18

Versión: LEFT JOIN y COALESCE son conceptos consolidados anteriores a PostgreSQL 18.

Lección 4 · Subconsultas correlacionadas con EXISTS

↑ Volver al índice

Idea principal

EXISTS responde si la subconsulta produce al menos una fila. Es muy útil para “clientes con pedidos” o “productos nunca vendidos”.

Ejemplo ejecutable

```
CREATE TEMP TABLE l4_clientes(id int PRIMARY KEY,nombre text);
CREATE TEMP TABLE l4_pedidos(id int PRIMARY KEY,cliente_id int REFERENCES l4_clientes);
INSERT INTO l4_clientes VALUES (1,'Ana'),(2,'Luis'),(3,'Marta');
INSERT INTO l4_pedidos VALUES (101,1),(102,2);

SELECT c.nombre
FROM l4_clientes c
WHERE EXISTS (
  SELECT 1 FROM l4_pedidos p
  WHERE p.cliente_id=c.id
)
ORDER BY c.id;
```

Resultado esperado

Comprobación: Ana
Luis

Ejercicio práctico

Devuelve los clientes que no han realizado pedidos.

Solución explicada

Cambia EXISTS por NOT EXISTS. Marta será el único resultado.

PostgreSQL 18

Versión: EXISTS y NOT EXISTS no son novedades de PostgreSQL 18.

Lección 5 · CTE con WITH para dividir consultas complejas

↑ Volver al índice

Idea principal

Una CTE da nombre a un resultado intermedio durante una consulta. Mejora la legibilidad y permite reutilizar cálculos como totales por pedido.

Ejemplo ejecutable

```
CREATE TEMP TABLE l5_lineas(pedido_id int,cantidad int,precio numeric(10,2));
INSERT INTO l5_lineas VALUES (101,2,50),(101,1,25),(102,1,300),(103,1,80);

WITH totales AS (
  SELECT pedido_id,SUM(cantidad*precio) AS total
  FROM l5_lineas
  GROUP BY pedido_id
)
SELECT pedido_id,total
FROM totales
WHERE total > (SELECT AVG(total) FROM totales)
ORDER BY pedido_id;
```

Resultado esperado

Comprobación: 102 | 300.00

Ejercicio práctico

Devuelve el pedido con el menor total utilizando la misma CTE.

Solución explicada

Ordena total ASC y usa LIMIT 1, o compara con MIN(total).

PostgreSQL 18

Versión: Las CTE existen desde versiones antiguas. PostgreSQL puede integrar CTE no materializadas en

el plan cuando es seguro.

Lección 29 · CTE recursiva para jerarquías

↑ Volver al índice

Idea principal

WITH RECURSIVE permite recorrer árboles de profundidad variable: categorías, organigramas o dependencias. Conviene protegerse contra ciclos.

Ejemplo ejecutable

```
CREATE TEMP TABLE l29_cat(id int PRIMARY KEY,nombre text,padre_id int REFERENCES l29_cat);
INSERT INTO l29_cat VALUES (1,'Electrónica',NULL),(2,'Ordenadores',1),(3,'Accesorios',1),
(4,'Portátiles',2),(5,'Teclados',3),(6,'Ratones',3);

WITH RECURSIVE arbol AS (
  SELECT id,nombre,padre_id,0 nivel,ARRAY[id] ruta FROM l29_cat WHERE padre_id IS NULL
  UNION ALL
  SELECT c.id,c.nombre,c.padre_id,a.nivel+1,a.ruta||c.id
  FROM l29_cat c JOIN arbol a ON c.padre_id=a.id
  WHERE NOT c.id=ANY(a.ruta)
)
SELECT repeat('  ',nivel)||nombre AS categoria,nivel,ruta
FROM arbol ORDER BY ruta;
```

Resultado esperado

Comprobación: Electrónica -> Ordenadores -> Portátiles; Electrónica -> Accesorios -> Teclados/Ratones.

Ejercicio práctico

Obtén todas las subcategorías de Accesorios y suma el stock de sus productos.

Solución explicada

Usa una CTE recursiva que empiece en id=3 y JOIN final con productos; suma el stock de todas las categorías alcanzadas.

PostgreSQL 18

Versión: Las CTE recursivas y CYCLE ya existían. PostgreSQL 18 mejora observabilidad de nodos relacionados en EXPLAIN.

Funciones de ventana

Este bloque agrupa las lecciones relacionadas para estudiar el tema de forma progresiva.

Lección 6 · ROW_NUMBER(), RANK() y PARTITION BY

↑ Volver al índice

Idea principal

Las funciones de ventana calculan sobre varias filas sin colapsarlas. ROW_NUMBER asigna una secuencia única; RANK conserva empates y deja huecos.

Ejemplo ejecutable

```
CREATE TEMP TABLE l6_pedidos(id int PRIMARY KEY,cliente text,total numeric(10,2));
INSERT INTO l6_pedidos VALUES (101,'Ana',125),(102,'Luis',300),(103,'Ana',80),(104,'Ana',125),
(105,'Luis',120);

SELECT id,cliente,total,
       ROW_NUMBER() OVER(PARTITION BY cliente ORDER BY total DESC,id) AS pos,
       RANK() OVER(PARTITION BY cliente ORDER BY total DESC) AS rango
FROM l6_pedidos
ORDER BY cliente,pos;
```

Resultado esperado

Comprobación: Ana: 125 -> pos 1/rango 1; 125 -> pos 2/rango 1; 80 -> pos 3/rango 3. Luis: 300 -> 1; 120 -> 2.

Ejercicio práctico

Obtén solo el pedido de mayor importe de cada cliente.

Solución explicada

Calcula ROW_NUMBER() en una CTE y filtra pos = 1 en la consulta exterior.

PostgreSQL 18

Versión: Las funciones de ventana no son nuevas en PostgreSQL 18.

Lección 7 · Marcos ROWS y acumulados

↑ Volver al índice

Idea principal

El marco delimita qué filas de una partición utiliza la función de ventana. ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW produce acumulados.

Ejemplo ejecutable

```
CREATE TEMP TABLE l7_ventas(id int,fecha date,total numeric(10,2));
INSERT INTO l7_ventas VALUES (1,'2026-10-15',100),(2,'2026-10-16',150),(3,'2026-10-17',80),
(4,'2026-10-18',220);
```

```
SELECT id,fecha,total,
       SUM(total) OVER(ORDER BY fecha,id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
acumulado,
       ROUND(AVG(total) OVER(ORDER BY fecha,id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW),2) AS
media3
FROM l7_ventas
ORDER BY fecha,id;
```

Resultado esperado

Comprobación: Acumulado: 100, 250, 330, 550. Media móvil de 3: 100, 125, 110, 150.

Ejercicio práctico

Calcula un acumulado independiente para cada cliente.

Solución explicada

Añade PARTITION BY cliente dentro de OVER(...).

PostgreSQL 18

Versión: ROWS, RANGE y GROUPS ya existían antes de PostgreSQL 18.

Lección 24 · Marcos GROUPS y EXCLUDE

↑ Volver al índice

Idea principal

GROUPS cuenta grupos de pares con el mismo ORDER BY. EXCLUDE permite quitar la fila actual, sus pares o el grupo completo del marco.

Ejemplo ejecutable

```
CREATE TEMP TABLE l24_pedidos(id int,cliente text,fecha date,total numeric(10,2));
INSERT INTO l24_pedidos VALUES (101,'Ana','2026-01-01',100),(102,'Ana','2026-01-01',50),
(103,'Ana','2026-01-02',80),(104,'Ana','2026-01-03',120);

SELECT id,fecha,total,
COALESCE(SUM(total) OVER(
  PARTITION BY cliente ORDER BY fecha
  GROUPS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  EXCLUDE GROUP
),0) AS anteriores
FROM l24_pedidos
ORDER BY fecha,id;
```

Resultado esperado

Comprobación: Para 1 de enero -> 0; 2 de enero -> 150; 3 de enero -> 230.

Ejercicio práctico

Compara GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW con ROWS BETWEEN 1 PRECEDING AND CURRENT ROW.

Solución explicada

GROUPS incluye el grupo completo de la fecha anterior; ROWS solo una fila física anterior.

PostgreSQL 18

Versión: GROUPS y EXCLUDE existen desde antes; PostgreSQL 18 mejora rendimiento/observabilidad de algunos agregados de ventana.

Índices, EXPLAIN y rendimiento

Este bloque agrupa las lecciones relacionadas para estudiar el tema de forma progresiva.

Lección 8 · Índices B-tree y lectura básica de EXPLAIN

↑ Volver al índice

Idea principal

Un índice puede reducir las filas y bloques que PostgreSQL necesita visitar, pero el planificador decide si usarlo. EXPLAIN muestra el plan estimado.

Ejemplo ejecutable

```
CREATE TEMP TABLE l8_pedidos(id int PRIMARY KEY,cliente_id int,fecha date,total
numeric(10,2));
INSERT INTO l8_pedidos
SELECT n,((n-1)%500)+1,DATE '2026-01-01'+(n%365),(10+n%990)::numeric(10,2)
FROM generate_series(1,20000) n;
ANALYZE l8_pedidos;

CREATE INDEX l8_cliente_fecha_idx ON l8_pedidos(cliente_id,fecha);
EXPLAIN (ANALYZE, BUFFERS)
SELECT id,fecha,total
FROM l8_pedidos
WHERE cliente_id=42 AND fecha>=DATE '2026-07-01'
ORDER BY fecha;
```

Resultado esperado

Comprobación: El plan puede usar Index Scan o Bitmap Index/Heap Scan. El plan exacto depende de estadísticas y costes.

Ejercicio práctico

¿Qué índice crearías para búsquedas frecuentes solo por fecha?

Solución explicada

CREATE INDEX ... ON l8_pedidos(fecha); aunque PostgreSQL 18 puede considerar skip scan en ciertos índices multicolumna, no sustituye siempre a un índice específico.

PostgreSQL 18

Versión: PostgreSQL 18 añade B-tree skip scan, útil en algunos casos donde falta una condición sobre la primera columna del índice.

Lección 21 · EXPLAIN ANALYZE: estimado vs real

↑ Volver al índice

Idea principal

EXPLAIN ANALYZE ejecuta la consulta y permite comparar rows estimadas con actual rows, además de tiempo, loops, buffers y memoria de planificación.

Ejemplo ejecutable

```
CREATE TEMP TABLE l21_pedidos(id int PRIMARY KEY,cliente_id int,fecha date,total
numeric(10,2));
INSERT INTO l21_pedidos
SELECT n,(n%100)+1,DATE '2026-01-01'+(n%365),(20+n%500)::numeric(10,2)
FROM generate_series(1,100000) n;
ANALYZE l21_pedidos;
CREATE INDEX l21_cliente_fecha_idx ON l21_pedidos(cliente_id,fecha);
EXPLAIN (ANALYZE, BUFFERS, MEMORY)
SELECT COUNT(*),SUM(total) FROM l21_pedidos
WHERE cliente_id=42 AND fecha>=DATE '2026-06-01' AND fecha<DATE '2026-07-01';
```

Resultado esperado

Comprobación: Debe aparecer un plan real. El nodo puede ser Index Scan o Bitmap. Compara rows vs actual rows y buffers.

Ejercicio práctico

Analiza cliente 73 durante septiembre y compara el plan.

Solución explicada

Cambia cliente y rango de fechas. La combinación cliente_id+fecha puede usar el mismo índice multicolumna.

PostgreSQL 18

Versión: PostgreSQL 18 mejora la observabilidad de EXPLAIN, incluida información adicional de memoria y búsquedas en índices.

Roles, permisos y seguridad

Este bloque agrupa las lecciones relacionadas para estudiar el tema de forma progresiva.

Lección 9 · Roles, permisos y mínimo privilegio

↑ Volver al índice

Idea principal

En PostgreSQL todo son roles. Los roles con LOGIN actúan como usuarios; los roles NOLOGIN son buenos grupos de permisos.

Ejemplo ejecutable

```
BEGIN;
CREATE ROLE l9_lectura NOLOGIN;
CREATE ROLE l9_empleado NOLOGIN;
GRANT l9_lectura TO l9_empleado;
CREATE SCHEMA l9_tienda;
CREATE TABLE l9_tienda.productos(id int PRIMARY KEY,nombre text);
GRANT USAGE ON SCHEMA l9_tienda TO l9_lectura;
GRANT SELECT ON l9_tienda.productos TO l9_lectura;

SELECT has_table_privilege('l9_empleado','l9_tienda.productos','SELECT') AS puede_leer,
       has_table_privilege('l9_empleado','l9_tienda.productos','INSERT') AS puede_insertar;
ROLLBACK;
```

Resultado esperado

Comprobación: puede_leer = true; puede_insertar = false.

Ejercicio práctico

Crea un grupo de ventas con INSERT, pero sin UPDATE ni DELETE.

Solución explicada

CREATE ROLE ... NOLOGIN; GRANT INSERT ON ... TO ...; después GRANT el grupo al rol del empleado.

PostgreSQL 18

Versión: PostgreSQL 18 añade pg_get_acl() para consultar ACL de objetos de forma más uniforme.

Lección 22 · Row-Level Security (RLS) en un SaaS

↑ Volver al índice

Idea principal

RLS filtra filas según políticas dentro de la base de datos. USING controla qué filas se ven/modifican; WITH CHECK valida nuevas filas.

Ejemplo ejecutable

```
CREATE SCHEMA l22;
CREATE TABLE l22.pedidos(id int PRIMARY KEY,tienda_id int,cliente text,total numeric(10,2));
```

```
ALTER TABLE l22.pedidos ENABLE ROW LEVEL SECURITY;
CREATE ROLE l22_tienda_1 NOLOGIN;
GRANT SELECT,INSERT,UPDATE ON l22.pedidos TO l22_tienda_1;
GRANT USAGE ON SCHEMA l22 TO l22_tienda_1;
CREATE POLICY tienda1 ON l22.pedidos TO l22_tienda_1
USING (tienda_id=1) WITH CHECK (tienda_id=1);
```

Resultado esperado

Comprobación: El rol solo puede ver/modificar filas con tienda_id=1 y no puede insertar filas para otra tienda.

Ejercicio práctico

Crea una política equivalente para tienda 2.

Solución explicada

Usa otra política TO l22_tienda_2 con USING/WITH CHECK tienda_id=2.

PostgreSQL 18

Versión: RLS no es nuevo. PostgreSQL 18 añade OAuth y continúa la transición hacia autenticación más moderna; RLS sigue siendo autorización a nivel de filas.

PL/pgSQL: funciones, procedimientos y triggers

Este bloque agrupa las lecciones relacionadas para estudiar el tema de forma progresiva.

Lección 10 · Funciones PL/pgSQL

↑ Volver al índice

Idea principal

Una función puede recibir parámetros, consultar datos y devolver un valor. SELECT ... INTO asigna el resultado a una variable local.

Ejemplo ejecutable

```
CREATE TEMP TABLE l10_lineas(pedido_id int,cantidad int,precio numeric(10,2));
INSERT INTO l10_lineas VALUES (101,2,50),(101,1,25),(102,1,300);

CREATE OR REPLACE FUNCTION pg_temp.total_pedido(p_id int)
RETURNS numeric(12,2) LANGUAGE plpgsql STABLE AS $$
DECLARE v_total numeric(12,2);
BEGIN
  SELECT COALESCE(SUM(cantidad*precio),0)::numeric(12,2) INTO v_total
  FROM l10_lineas WHERE pedido_id=p_id;
  RETURN v_total;
END;$$;

SELECT pg_temp.total_pedido(101),pg_temp.total_pedido(999);
```

Resultado esperado

Comprobación: 125.00 | 0.00

Ejercicio práctico

Crea una función total_con_descuento(pedido, porcentaje) que valide 0–100.

Solución explicada

Usa IF para validar, llama a total_pedido y devuelve ROUND(total * (1 - descuento/100), 2).

PostgreSQL 18

Versión: PL/pgSQL, STABLE, variables y SELECT INTO ya existían antes de PostgreSQL 18.

Lección 11 · Procedimientos almacenados con CALL

↑ Volver al índice

Idea principal

Los procedimientos representan operaciones de negocio y se ejecutan con CALL. Pueden modificar varias tablas y, en ciertos contextos, controlar transacciones.

Ejemplo ejecutable

```
CREATE TEMP TABLE l11_pedidos(id int PRIMARY KEY,total numeric(12,2),estado text DEFAULT
'pendiente');
CREATE TEMP TABLE l11_pagos(id int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,pedido_id
int,importe numeric(12,2));
INSERT INTO l11_pedidos(id,total) VALUES (101,125),(102,300);

CREATE OR REPLACE PROCEDURE pg_temp.registrar_pago(p_id int,p_importe numeric)
LANGUAGE plpgsql AS $$
DECLARE v_total numeric; v_pagado numeric;
BEGIN
  SELECT total INTO v_total FROM l11_pedidos WHERE id=p_id;
  IF NOT FOUND THEN RAISE EXCEPTION 'Pedido no existe'; END IF;
  SELECT COALESCE(SUM(importe),0) INTO v_pagado FROM l11_pagos WHERE pedido_id=p_id;
  IF v_pagado+p_importe>v_total THEN RAISE EXCEPTION 'Pago supera saldo'; END IF;
  INSERT INTO l11_pagos(pedido_id,importe) VALUES(p_id,p_importe);
  IF v_pagado+p_importe=v_total THEN UPDATE l11_pedidos SET estado='pagado' WHERE id=p_id; END
IF;
END;$$;
CALL pg_temp.registrar_pago(101,125);
```

Resultado esperado

Comprobación: El pedido 101 queda con estado pagado.

Ejercicio práctico

Crea un procedimiento para marcar enviado solo si el pedido está pagado.

Solución explicada

UPDATE ... WHERE id=p_id AND estado='pagado'; IF NOT FOUND THEN RAISE EXCEPTION ... END IF;

PostgreSQL 18

Versión: CALL y procedimientos no son novedades de PostgreSQL 18.

Lección 12 · Triggers para controlar el stock

↑ Volver al índice

Idea principal

Un trigger ejecuta una función automáticamente ante INSERT/UPDATE/DELETE. NEW representa la fila nueva y OLD la fila anterior.

Ejemplo ejecutable

```
CREATE TEMP TABLE l12_productos(id int PRIMARY KEY,nombre text,stock int CHECK(stock>=0));
CREATE TEMP TABLE l12_lineas(id int PRIMARY KEY,producto_id int REFERENCES
l12_productos,cantidad int CHECK(cantidad>0));
INSERT INTO l12_productos VALUES (1,'Teclado',5),(2,'Ratón',2);

CREATE OR REPLACE FUNCTION pg_temp.descontar_stock() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  UPDATE l12_productos
  SET stock=stock-NEW.cantidad
```

```
  WHERE id=NEW.producto_id AND stock>=NEW.cantidad;
  IF NOT FOUND THEN RAISE EXCEPTION 'Stock insuficiente'; END IF;
  RETURN NEW;
END;$$;

CREATE TRIGGER l12_stock_trg BEFORE INSERT ON l12_lineas
FOR EACH ROW EXECUTE FUNCTION pg_temp.descontar_stock();

INSERT INTO l12_lineas VALUES (1,1,2);
SELECT * FROM l12_productos ORDER BY id;
```

Resultado esperado

Comprobación: Teclado queda con stock 3; Ratón sigue con 2.

Ejercicio práctico

Crea un trigger AFTER DELETE que reponga OLD.cantidad al stock.

Solución explicada

UPDATE productos SET stock=stock+OLD.cantidad WHERE id=OLD.producto_id; RETURN OLD;

PostgreSQL 18

Versión: Triggers, NEW y OLD son funcionalidades anteriores a PostgreSQL 18.

Funciones y modelado específicos de PostgreSQL 18

Este bloque agrupa las lecciones relacionadas para estudiar el tema de forma progresiva.

Lección 16 · Columnas generadas virtuales

↑ Volver al índice

Idea principal

Una columna generada virtual se calcula al leer la fila y no almacena físicamente el valor derivado.

Ejemplo ejecutable

```
CREATE TEMP TABLE l16_lineas(
  id int PRIMARY KEY,
  cantidad int,
  precio numeric(10,2),
  descuento numeric(5,2) DEFAULT 0,
  subtotal numeric(12,2) GENERATED ALWAYS AS (ROUND(cantidad*precio*(1-descuento/100),2))
VIRTUAL
);
INSERT INTO l16_lineas(id,cantidad,precio,descuento) VALUES (1,2,50,0),(2,1,25,10);
SELECT * FROM l16_lineas ORDER BY id;
```

Resultado esperado

Comprobación: Fila 1 subtotal 100.00; fila 2 subtotal 22.50.

Ejercicio práctico

Añade una columna virtual total_con_impuesto con un factor 1.08.

Solución explicada

ALTER TABLE ... ADD COLUMN ... GENERATED ALWAYS AS
(ROUND(cantidad*precio*(1-descuento/100)*1.08,2)) VIRTUAL;

PostgreSQL 18

Versión: Novedad de PostgreSQL 18: columnas generadas VIRTUAL y opción virtual por defecto cuando no se especifica STORED.

Lección 17 · UUIDv7: identificadores ordenables por tiempo

↑ Volver al índice

Idea principal

UUIDv7 incorpora información temporal y es útil como clave distribuida con mejor localidad que UUIDv4.

Ejemplo ejecutable

```
CREATE TEMP TABLE l17_pedidos(id uuid PRIMARY KEY DEFAULT uuidv7(),cliente text,total
numeric(10,2));
INSERT INTO l17_pedidos(cliente,total) VALUES ('Ana',125),('Luis',300),('Marta',80);
SELECT cliente,uuid_extract_version(id) AS version,uuid_extract_timestamp(id) AS momento
```

```
FROM l17_pedidos
ORDER BY id;
```

Resultado esperado

Comprobación: Todos los UUID tendrán version = 7. Las marcas temporales serán próximas al momento de inserción.

Ejercicio práctico

Genera UUIDv7 desplazados -2 days, -1 day y actual; ordénalos por UUID.

Solución explicada

Usa uuidv7(interval) y ORDER BY id. El orden reflejará aproximadamente la secuencia temporal.

PostgreSQL 18

Versión: Novedad de PostgreSQL 18: uuidv7(), uuidv4() y funciones de extracción asociadas.

Lección 18 · Periodos sin solapamiento con WITHOUT OVERLAPS

↑ Volver al índice

Idea principal

PostgreSQL 18 permite expresar claves temporales que prohíben periodos solapados para un mismo identificador.

Ejemplo ejecutable

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TEMP TABLE l18_precios(
  producto_id int NOT NULL,
  vigencia daterange NOT NULL,
  precio numeric(10,2) NOT NULL,
  PRIMARY KEY(producto_id,vigencia WITHOUT OVERLAPS)
);
INSERT INTO l18_precios VALUES
(1,daterange('2026-01-01','2026-04-01','[)'),300),
(1,daterange('2026-04-01','2026-07-01','[)'),280);
SELECT * FROM l18_precios WHERE vigencia @> DATE '2026-05-15';
```

Resultado esperado

Comprobación: Se devuelve el periodo 2026-04-01 a 2026-07-01 con precio 280.

Ejercicio práctico

Intenta insertar un periodo 2026-03-15 a 2026-05-01 para el mismo producto.

Solución explicada

La restricción debe rechazarlo porque se solapa con periodos existentes.

PostgreSQL 18

Versión: Novedad de PostgreSQL 18: sintaxis WITHOUT OVERLAPS en claves UNIQUE/PRIMARY KEY.

Lección 19 · Claves foráneas temporales con PERIOD

↑ Volver al índice

Idea principal

PERIOD permite exigir que un intervalo de la tabla hija esté completamente cubierto por intervalos válidos de la tabla padre.

Ejemplo ejecutable

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TEMP TABLE l19_disp(
 producto_id int, periodo daterange NOT NULL,
 PRIMARY KEY(producto_id,periodo WITHOUT OVERLAPS)
);
INSERT INTO l19_disp VALUES
(1,daterange('2026-01-01','2026-04-01','[)')),(1,daterange('2026-04-01','2026-07-01','[)'));
CREATE TEMP TABLE l19_promo(
 id int PRIMARY KEY, producto_id int, periodo daterange,
 FOREIGN KEY(producto_id,PERIOD periodo) REFERENCES l19_disp(producto_id,PERIOD periodo)
);
INSERT INTO l19_promo VALUES (101,1,daterange('2026-03-01','2026-05-01','[)'));
```

Resultado esperado

Comprobación: La promoción se acepta porque la unión de los periodos padre cubre completamente marzo-mayo.

Ejercicio práctico

Crea un hueco en disponibilidad e intenta insertar una promoción que lo atraviese.

Solución explicada

La clave foránea temporal debe rechazarla con foreign_key_violation.

PostgreSQL 18

Versión: Novedad de PostgreSQL 18: claves foráneas temporales con PERIOD.

Lección 25 · COPY tolerante a errores: REJECT_LIMIT

↑ Volver al índice

Idea principal

COPY puede ignorar errores de conversión con ON_ERROR ignore. PostgreSQL 18 permite limitar cuántas filas erróneas se toleran con REJECT_LIMIT.

Ejemplo ejecutable

```
CREATE TEMP TABLE l25_productos(id int PRIMARY KEY,nombre text,precio numeric(10,2),stock
int);
-- En psql:
COPY l25_productos(id,nombre,precio,stock) FROM STDIN
WITH (FORMAT csv,HEADER true,ON_ERROR ignore,REJECT_LIMIT 1,LOG_VERBOSITY silent);
id,nombre,precio,stock
1,Monitor,299.90,10
2,Teclado,no_es_numero,15
3,Ratón,25.50,30
\.
SELECT * FROM l25_productos ORDER BY id;
```

Resultado esperado

Comprobación: Se importan Monitor y Ratón; la fila del Teclado se descarta por error de conversión.

Ejercicio práctico

Repite con dos errores y REJECT_LIMIT 2.

Solución explicada

Mientras no se supere el límite, las filas válidas se cargan. CHECK, FK o duplicados no son errores de conversión ignorables automáticamente.

PostgreSQL 18

Versión: Novedad de PostgreSQL 18: REJECT_LIMIT y LOG_VERBOSITY silent para COPY con errores ignorables.

Lección 26 · Vistas materializadas para informes

↑ Volver al índice

Idea principal

Una vista materializada almacena físicamente el resultado de una consulta. REFRESH recalcula el contenido; CONCURRENTLY permite refrescar manteniendo lecturas.

Ejemplo ejecutable

```
CREATE TABLE l26_lineas(producto text,cantidad int,precio numeric(10,2));
INSERT INTO l26_lineas VALUES ('Monitor',3,290),('Teclado',2,50),('Ratón',3,25);
CREATE MATERIALIZED VIEW l26_resumen AS
SELECT producto,SUM(cantidad) unidades,SUM(cantidad*precio)::numeric(12,2) ventas
FROM l26_lineas GROUP BY producto;
CREATE UNIQUE INDEX l26_resumen_uq ON l26_resumen(producto);
SELECT * FROM l26_resumen ORDER BY producto;
```

Resultado esperado

Comprobación: Monitor, Ratón y Teclado aparecen con sus unidades/ventas agregadas.

Ejercicio práctico

Inserta una venta nueva, comprueba que la vista no cambia y ejecuta REFRESH MATERIALIZED VIEW CONCURRENTLY.

Solución explicada

Después del REFRESH el resumen se actualiza. CONCURRENTLY exige un índice UNIQUE adecuado.

PostgreSQL 18

Versión: Novedad de PostgreSQL 18 relacionada: COPY TO puede usar directamente una vista materializada poblada.

Lección 27 · Búsquedas Unicode con casefold()

↑ Volver al índice

Idea principal

casefold() realiza un plegado Unicode más completo que lower(), útil para equivalencias como Straße y STRASSE.

Ejemplo ejecutable

```
SELECT lower('Straße' COLLATE pg_unicode_fast) AS lower_val,
       casefold('Straße' COLLATE pg_unicode_fast) AS folded;

CREATE TEMP TABLE l27_productos(id int PRIMARY KEY,nombre text);
INSERT INTO l27_productos VALUES (1,'Straße USB-Adapter'),(2,'STRASSE Netzteil'),(3,'Büro-
Monitor');
SELECT * FROM l27_productos
WHERE casefold(nombre COLLATE pg_unicode_fast) LIKE '%'||casefold('strasse' COLLATE
pg_unicode_fast)||'%';
```

Resultado esperado

Comprobación: casefold convierte Straße a strasse y la búsqueda encuentra las filas 1 y 2.

Ejercicio práctico

Busca BÜRO sin distinguir mayúsculas/minúsculas.

Solución explicada

Aplica casefold() tanto a la columna como al parámetro de búsqueda.

PostgreSQL 18

Versión: Novedad de PostgreSQL 18: casefold() y la intercalación integrada pg_unicode_fast.

Lección 28 · Búsqueda aproximada con pg_trgm

↑ Volver al índice

Idea principal

pg_trgm compara trigramas y permite búsquedas parciales o tolerantes a errores tipográficos. Puede usar índices GIN o GiST.

Ejemplo ejecutable

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE TABLE l28_productos(id int PRIMARY KEY,nombre text);
INSERT INTO l28_productos VALUES (1,'Büro-Monitor'),(2,'Monitor 27 Zoll'),(3,'Monitorkabel
HDMI'),(4,'Gaming-Maus');
CREATE INDEX l28_trgm_idx ON l28_productos USING gin((casefold(nombre COLLATE
pg_unicode_fast)) gin_trgm_ops);
SELECT id,nombre
FROM l28_productos
WHERE word_similarity(casefold('monotor' COLLATE pg_unicode_fast),casefold(nombre COLLATE
pg_unicode_fast))>=0.40
ORDER BY id;
```

Resultado esperado

Comprobación: Deben aparecer los tres nombres relacionados con monitor; el umbral puede necesitar ajuste con datos reales.

Ejercicio práctico

Compara el plan de LIKE %monitor% con y sin el índice trigramático en una tabla grande.

Solución explicada

Usa EXPLAIN (ANALYZE,BUFFERS). En tablas pequeñas un Seq Scan puede seguir siendo correcto.

PostgreSQL 18

Versión: pg_trgm no es nuevo; en PostgreSQL 18 se combina bien con casefold() y pg_unicode_fast.

Apéndice · Ruta de estudio recomendada

Orden sugerido si quieres repetir el curso desde cero:

Fundamentos SQL: Lección 2, Lección 3, Lección 4, Lección 5

Ventanas: Lección 6, Lección 7, Lección 24

Rendimiento: Lección 8, Lección 21

Seguridad: Lección 9, Lección 22

PL/pgSQL: Lección 10, Lección 11, Lección 12

Administración: Lección 1, Lección 13, Lección 14, Lección 15, Lección 23

Funciones PostgreSQL 18: Lección 16, Lección 17, Lección 18, Lección 19, Lección 25, Lección 26, Lección 27, Lección 28

Jerarquías: Lección 29

Buenas prácticas generales

- Prueba primero en una base de desarrollo; EXPLAIN ANALYZE ejecuta realmente la sentencia.
- Usa transacciones para cambios que afecten varias tablas.
- No desactives autovacuum globalmente salvo que sepas exactamente por qué.
- Concede permisos a roles de grupo y aplica mínimo privilegio.
- No crees índices “por si acaso”: mide con EXPLAIN (ANALYZE, BUFFERS).
- Parametriza entradas de usuario ($1, $2, ...) y evita concatenar SQL.
- Cuando una característica sea específica de PostgreSQL 18, comprueba SHOW server_version antes de probarla.